Whole Pipeline

49000 image → 490 batch × 100 image (or sample)

↳ 1 batch = 1 iteration

490 iteration × 10 epoch = 4900 iteration ↩

3072 → 100 → 100 → 10 (input → hidden1 → hidden2 → classes)

Code Map (1 file = CONFIG + 8 PARTS)

CONFIG

Colab ? → Google Drive path : → folder of this file

DATASET_DIR , CIFAR10_DIR , SAVE_DIR

PART 1 , layers.py → every layer (forward + backward)

affine : out = $x \cdot W + b$

relu : max(0, x)

batchnorm : $(x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} \rightarrow \gamma \cdot \hat{x} + \beta$ (+ backward_alt)

layernorm : same, but per image (not per batch)

dropout : x × mask / p

conv , max_pool , spatial norms → empty TODO , not used

loss : svm_loss , softmax_loss

PART 2 , layer_utils.py

affine_relu = affine → relu ("sandwich", forward + backward)

PART 3 , optim.py → update rules

sgd , sgd_momentum , rmsprop , adam → OPTIM_RULES (name → function)

PART 4 , fc_net.py → models

TwoLayerNet (not used in main)

TwoLayerNet (not used in main)

FullyConnectedNet → __init__ , loss , save , load

PART 5 , solver.py → training loop + ACCURACY

__init__ , _reset , _step , _save_checkpoint , check_accuracy , train

PART 6 , data_utils.py → load data

load_pickle , load_CIFAR_batch , load_CIFAR10 , get_CIFAR10_data

tiny imagenet , load_models , imagenet_val → not used

PART 7 , gradient_check.py → numerical gradient

grad ≈ (f(x+h) − f(x−h)) / 2h (centered , h = 1e-5)

grad_check_sparse → relative error = |num − ana| / (|num| + |ana|)

PART 8 , main (notebook) → Step 1 ... Step 8 اجرا

PART 1–7 only define functions , PART 8 runs them

1, Setup & Data Preparation

1.1 Environment setup & import

Colab ? → mount Google Drive

plt settings : figsize 10×8 , cmap gray

1.3 Download Dataset (CIFAR-10)

already there ? → skip : → download tar.gz → extract

1.4 Load Dataset → get_CIFAR10_data()

5 batch × 10000 = 50000 train + 10000 test

reshape → (10000, 32, 32, 3)

split data → 49000 train , 1000 val , 1000 test

Normalize → − mean image

transpose → (N, 3, 32, 32) (channel first)

1.5 Prepare & Create Data

Reshape → (N, 3072) (32×32×3 = 3072)

− mean image again

small_data → 50 image (overfit test)

full_data → 49000 train + 1000 val

2, Create Model & Initialize Hyperparameters

(runs once)

2.1 Define Architecture

{ affine → batchnorm → relu → dropout } × 2 → affine → softmax

2.2 Layer Dimensions

dims = [3072 , 100 , 100 , 10] → num_layers = 3

W1 : 3072×100 , W2 : 100×100 , W3 : 100×10

2.3 Initialize Parameters

W = 0.05 × random , b = 0

$\gamma = 1$, $\beta = 0$ (hidden layers only)

dropout_param → mode = train , p = 0.9 (keep)

bn_params → 2 dict (1 per hidden layer) , running mean / var

reg = 0.01 , dtype = float64

Hyperparameters ابزار ⇒ W , b , γ , β , reg , dropout , lr , lr decay , epoch , batch

3, Training Setup & Mini-Batch

3.1 Select training data → full_data (small_data → overfit check)

3.2 Training Hyperparameters

lr = 0.001 , batch = 100 , epoch = 10

3.3 Create Solver & initialize

update rule = adam , lr decay = 0.95 , print every 100

_reset → epoch = 0 , best val acc = 0 , histories = []

optim_configs → 1 config per parameter (W1 , b1 , γ1 ...)

solver.train() → loop آغاز ⟳

iteration per epoch = 49000 / 100 = 490

num iterations = 490 × 10 = 4900

Train Model → loop ⟳ (4900 ×)

3.4 Start Mini-Batch → _step()

100 random image → X_batch , y_batch

4, Forward Pass (model.loss(X, y) → train mode)

4.1 Prepare Input → out = X , mode = train

4.2 Hidden Layer Forward (i = 1 , 2)

affine → $a = out \cdot W + b$

batchnorm → μ , σ^2 of this batch

$\hat{x} = (a - \mu) / \sqrt{(\sigma^2 + \varepsilon)} \rightarrow \gamma \cdot \hat{x} + \beta$

running mean / var = 0.9 · old + 0.1 · new

ReLU → max(0, a)

dropout → mask = (random < 0.9) / 0.9 → out × mask

save cache (for backward) → out (100 × 100)

4.3 Final Output Layer → affine only (no relu, no norm, no dropout)

4.4 Scores → (100 × 10)

5, Loss Calculation

5.1 Softmax

scores − max (stability) → exp → / sum → probs

each row sum = 1

5.2 Correct-class probs → probs[i, y_i]

5.3 data loss = $-\sum \log(p \text{ correct}) / N$

5.4 reg loss = $0.5 \cdot reg \cdot \sum (W1^2 + W2^2 + W3^2)$

5.5 total loss = data loss + reg loss

dout = (probs − 1 for correct class) / N (100 × 10)

loss → loss_history

6, Backward Pass & Gradient (chain rule)

6.1 Receive dout from softmax

6.2 Final Affine Backward

$dx = dout \cdot W^T$, $dW = x^T \cdot dout$, $db = \sum dout$

dW3 + reg · W3

6.3 Hidden Layers Backward (reverse: i = 2, 1)

dropout ← dout × mask

ReLU ← dout, 0 where a ≤ 0

batchnorm ← $d\gamma = \sum dout \cdot \hat{x}$, $d\beta = \sum dout$, dx

affine ← dW + reg · W, db, dx → layer below

6.4 All Gradients Ready

W1 b1 γ1 β1, W2 b2 γ2 β2, W3 b3

7. Optimizer & Parameter Update

7.1 Receive w, dw, config (each parameter → own config)

7.2 Choose Update Rule

sgd → $w - lr \cdot dw$

momentum → $v = 0.9\,v - lr \cdot dw$ → $w + v$

rmsprop → $cache = 0.99\,cache + 0.01\,dw^2$ → $w - lr \cdot dw / \sqrt{cache}$

adam ✓ (used here)

7.3 Compute Next Parameters (adam)

$t + 1$

$m = 0.9\,m + 0.1\,dw$ (momentum)

$v = 0.999\,v + 0.001\,dw^2$ (rmsprop)

$\hat{m} = m / (1 - 0.9^t)$, $\hat{v} = v / (1 - 0.999^t)$ (bias correction)

$w = w - lr \cdot \hat{m} / (\sqrt{\hat{v}} + 1e\text{-}8)$

7.4 Update Model Parameters

After each iteration

every 100 iteration → print loss

end of epoch ? (every 490 iteration)

→ epoch + 1

→ lr × 0.95 (lr decay)

first / last iteration or end of epoch ?

→ train acc (1000 sample) & val acc (1000)

→ save to history , checkpoint (off)

→ val acc > best ? → save best W

↩ back to 3.4 ... until 4900

loop end → model = best W

8, Evaluation → ACCURACY

8.1 plot → train acc vs val acc (per epoch)

8.2 test acc = check_accuracy(X_test)

8.3 print → test acc , best val acc

check_accuracy()

(too many ? → random sample)

split → batches of 100

model.loss(X) , no y → test mode

batchnorm → running mean / var (not batch)

dropout → off

scores → argmax → predicted class

accuracy = mean(y_pred == y)

Steps 4 → 7 by hand (1 iteration)

seed 231 → 100 random image → X , y

mode = train (dropout + batchnorm)

4.1 → 4.4 → 5.1 → 5.5 → 6.1 → 6.4 → 7.1 → 7.4

print shapes : out 100×100 , scores 100×10 , dout 100×10

only to show inside the loop

Notes

- no Step 1.2 in the code (1.1 → 1.3)